

Progetto S2/L5

Obiettivo: L'obiettivo dell'esercitazione di oggi consiste nel identificare che tipo di script sta eseguendo il programma in Fig.1 e di individuarne gli errori, suggerendo una soluzione per ognuno di essi.

Fig.1 - Codice Sorgente Analizzato

```
1  import datetime
2
3  while True
4      comando_utente = input("Cosa vuoi sapere? ")
5      if comando_utente == "esci":
6          print("Arrivederci!")
7          break
8      else:
9          print(assistente_virtuale(comando_utente))
10
11 def assistente_virtuale(comando):
12     if comando == "Qual è la data di oggi?":
13         oggi = datetime.datetime.today()
14         risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15     elif comando == "Che ore sono?":
16         ora_attuale = datetime.datetime.now().time()
17         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18     elif comando == "Come ti chiami?":
19         risposta = "Mi chiamo Assistente Virtuale"
20     else:
21         risposta = "Non ho capito la tua domanda."
22     return risposta
23
```

Analisi del codice (Step-by-Step)

L'obiettivo dello script è programmare una logica di Intelligenza Artificiale di base capace di interrogare il sistema operativo per ottenere informazioni su data e tempo.

Riga 1 — Preparazione del modulo

L'esecuzione si apre con il caricamento del pacchetto `datetime` dalla libreria standard. Questo passaggio è propedeutico a ogni altra operazione, poiché fornisce gli strumenti necessari per interrogare il sistema sulle informazioni di data e ora.

Riga 3 — Ciclo di vita dell'applicazione

Viene instaurato un loop infinito tramite il comando `while True`. Tale scelta strutturale serve a garantire che il software rimanga attivo e in ascolto costante, impedendo la chiusura immediata e permettendo all'utente di formulare richieste in modo continuativo finché non decide di interrompere il flusso con un `break`.

Riga 4 — Acquisizione della richiesta utente

Il programma utilizza l'istruzione `input()` per presentare la domanda 'Cosa vuoi sapere?' e sospendere l'elaborazione. In questo istante, il sistema attende l'inserimento testuale dell'utente, memorizzando poi la stringa digitata nella variabile `comando_utente` per le successive analisi.

Righe 5-9 — Logica di smistamento e gestione chiusura

In questo blocco viene definita la biforcazione principale: se l'input intercettato è 'esci' (riga 5), il software stampa un messaggio di congedo (riga 6) e dovrebbe procedere allo stop del ciclo. Negli altri casi (else riga 8), la richiesta viene passata alla funzione `assistente_virtuale()` per l'elaborazione del contenuto (riga 9).

Riga 11 — Definizione della logica decisionale

Viene dichiarata la funzione `assistente_virtuale(comando)`, il modulo deputato a ricevere l'input e generare la risposta corrispondente. Nota: il fatto che la funzione sia scritta dopo la riga che la richiama è un errore di posizionamento che vedremo nei dettagli tecnici.

Righe 12-22 — Elaborazione e ritorno della risposta

La funzione gestisce le varie casistiche (data, ora, identità) costruendo una variabile locale `risposta`. Una volta identificato il caso corretto o gestito l'input ignoto (riga 20), il valore finale viene inviato indietro (return riga 22) verso il blocco principale del programma per essere mostrato a schermo.

Errori di Sintassi

Riga	Descrizione Errore	Soluzione Corretta
Riga 3	Mancanza dei due punti ':' dopo il <code>while True</code> .	Aggiungere ':' alla fine della riga.
Riga 7	Indentazione errata del comando <code>break</code> .	Spostare il <code>break</code> allineandolo all'if.
11-22	Definizione della funzione dopo la chiamata.	Spostare la def prima del ciclo <code>while</code> .

Errori di Logica e Semantica

Riga	Descrizione Errore	Soluzione Corretta
Riga 4	Mancanza di normalizzazione dell'input.	<code>comando_utente.lower().strip()</code>
Riga 13	Metodo inesistente (<code>datetime.datetoday</code>).	<code>datetime.date.today()</code>
Riga 16	Richiamo orario incompleto (manca <code>.time()</code>).	<code>datetime.datetime.now().time()</code>
Riga 22	Variabile 'risposta' non inizializzata.	<code>risposta = ""</code> all'inizio della funzione.

Denotazioni e Approfondimento Errori

Riga 3: Dopo `while True` non ci sono i due punti `:`. Senza di essi l'interprete non riconosce l'inizio del blocco e il codice non viene eseguito; è l'errore più bloccante in assoluto. Impedisce il caricamento dell'intero script: nessuna riga viene mai eseguita finché questo non viene risolto.

Spostamento Funzione: La funzione deve essere definita prima di essere chiamata in riga 3. Python legge il codice dall'alto verso il basso; se chiami una funzione prima di definirla, ricevi un `NameError`. Questo non è un errore sintattico puro ma di ordine di esecuzione (Runtime).

Riga 7: Il `break` è fuori dall'indentazione dell'if. È un errore strutturale: deve stare all'interno del blocco if per fermare il ciclo solo quando l'utente digita "esci". Se lasciato così, il programma si chiuderebbe al primo comando, indipendentemente dal testo inserito.

Riga 13: Analizzando la libreria `datetime`, il metodo `datetime.today()` non esiste. Si tratta di un errore di API che comporta l'interruzione del programma (crash) perché prova a compiere un'operazione impossibile per l'oggetto richiamato.

Riga 16: Controllando la libreria online si evince che c'è un altro errore: `datetime.datetime.now()` senza `.time()` finale. Sbagliare il richiamo genera un `AttributeError`. Stai chiedendo alla libreria qualcosa che non possiede o che è scritto nel modo sbagliato.

Case-Sensitivity: Se l'utente scrive "Esci" (con la maiuscola) e il codice controlla se `comando == "esci"`, il test fallirà. L'aggiunta di `.lower()` uniforma il pensiero della macchina, rendendo lo script molto più robusto e "user-friendly".

Riga 22 (Inizializzazione): La variabile 'risposta' viene creata solo dentro i rami `if`. Se per assurdo nessuno dei rami venisse eseguito, il `return` causerebbe un crash. Inizializzarla all'inizio è una "best practice" fondamentale per la manutenibilità futura del codice.

Dettaglio Soluzioni Proposte

Correzione Sintattica (Due Punti): L'inserimento dei due punti alla fine di `while True:` permette all'interprete di identificare l'inizio di un blocco di istruzioni. È il requisito fondamentale per stabilire la gerarchia del codice in Python.

Riorganizzazione Strutturale (Hoisting): Spostando la definizione della funzione `assistente_virtuale` all'inizio dello script, garantiamo che Python memorizzi il comando prima che venga effettivamente eseguito nel ciclo interattivo, eliminando il rischio di errori di nome.

Normalizzazione Input (Lower/Strip): Applicando `.lower().strip()` alla variabile dell'input, rendiamo il programma immune a variazioni di maiuscole o spazi superflui. Questa soluzione garantisce che "ESCI", " esci " e "Esci" vengano tutti riconosciuti correttamente come comando di chiusura.

Riallineamento Indentazione: Spostando il comando `break` all'interno del blocco `if`, ripristiniamo la logica di controllo corretta. In questo modo, l'uscita dal ciclo avviene esclusivamente quando la condizione di uscita è soddisfatta, e non in modo indiscriminato.

Aggiornamento Chiamate API: Sostituendo i metodi inesistenti con `datetime.date.today()` e `datetime.datetime.now().time()`, allineiamo lo script alla documentazione ufficiale di Python. Questo assicura che il programma possa effettivamente estrarre i dati temporali dal sistema operativo senza generare eccezioni.

Inizializzazione Variabili: Definire `risposta = ""` all'inizio della funzione protegge lo script da scenari in cui nessuna condizione viene soddisfatta, garantendo che il valore restituito sia sempre un oggetto valido e definito.

Conclusioni

Dall'analisi effettuata emerge che gli errori riscontrati sono riconducibili a carenze di forma, struttura e corretta applicazione delle API. In Python, la pulizia visiva non è un dettaglio estetico: la mancanza di punteggiatura sintattica (come i due punti) o un'indentazione imprecisa rendono il codice letteralmente illeggibile per la

macchina, interrompendo l'esecuzione prima ancora del suo inizio.

L'ordine logico è un altro pilastro fondamentale: definire le funzioni prima del loro utilizzo non è solo una convenzione, ma una necessità per permettere all'interprete di mappare correttamente gli oggetti in memoria. Inoltre, l'utilizzo di moduli esterni come `date` e `time` richiede una precisione assoluta nei percorsi dei metodi; anche un piccolo refuso trasforma un comando legittimo in un errore di attributo bloccante.

In definitiva, per trasformare questo script in un software funzionante e affidabile, occorre agire su due fronti: correggere la grammatica sintattica per permettere l'avvio e riorganizzare l'architettura semantica per garantirne la stabilità logica. Questo approccio non risolve solo i bug odierni, ma pone le basi per una manutenibilità futura, facilitando l'eventuale aggiunta di nuove funzionalità all'assistente virtuale.